

Système de Réponse aux Urgences Urbaines (SRUU)

Implémentation JADE Projet M1 Génie Logiciel

Équipe : DJEMIL HANA
Responsable : REFAS IKRAM
Année Universitaire 20252026

3 mai 2026

Table des matières

1	Résumé	3
2	Introduction	3
2.1	Contexte	3
2.2	Objectifs du SRUU	3
2.3	Contraintes techniques	3
3	Architecture du système	3
3.1	Vue d'ensemble	3
3.2	Description des agents	4
4	Protocoles de communication et ontologie	4
4.1	FIPA-ACL et Contract Net	4
4.2	Ontologie de domaine : EmergencyOntology	4
4.3	Découverte de services (DF)	5
5	Fonction d'utilité et logique de décision	5
5.1	Formulation mathématique	5
5.2	Algorithme de sélection	5
6	Comportement des agents et machines à états finis (FSM)	6
6.1	Behaviours JADE	6
6.2	Machines à états	6
6.3	Mouvement incrémental	6
6.4	Bootstrap séquentiel	6
7	Robustesse et gestion des conflits	7
8	Résultats et évaluation	7
8.1	Scénarios de test	7
8.2	Métriques de performance	7

9	Discussion et évaluation qualitative	7
9.1	Forces	7
9.2	Limites	8
9.3	Pistes damélioration	8
10	Conclusion	8
A	Annexe : Questions/Réponses pour la soutenance	8
B	Glossaire	9
C	Bibliographie	9

1 Résumé

Ce rapport présente le SRUU (Système de Réponse aux Urgences Urbaines), une simulation multi-agents développée avec JADE. Le système coordonne 9 types d'agents autonomes (capteurs, répartiteur, ambulances, pompiers, police, BCU, coordinateur médical, contrôleur de trafic, enregistreur) pour gérer des incidents urbains (incendies, crises médicales, effondrements, biohazards, fuites cryogéniques) sur une grille 50x50. L'architecture repose sur le protocole FIPA Contract Net pour l'affectation dynamique, une ontologie de domaine formalisant les communications ACL, et une fonction d'utilité multi-critères optimisant la sélection des unités.

2 Introduction

2.1 Contexte

Les systèmes multi-agents (SMA) modélisent des environnements complexes où des entités autonomes interagissent pour résoudre des problèmes distribués. Dans la gestion de crise urbaine, la coordination en temps réel de services d'urgence (pompiers, police, médical) représente un défi majeur.

2.2 Objectifs du SRUU

- Détection automatisée par capteurs distribués.
- Coordination intelligente via négociation Contract Net.
- Gestion simultanée de 5 types d'incidents.
- Robustesse : pas d'attente active, gestion des conflits, réaffectation dynamique.
- Audit complet avec métriques de performance.

2.3 Contraintes techniques

- Framework JADE exclusivement.
- Communication FIPA-ACL (INFORM, REQUEST, CFP, PROPOSE, ACCEPT_PROPOSAL, REJECT_PROPOSAL, FAILURE).
- Découverte de services via DF (pas de noms codés en dur).
- Pas d'attente active (uniquement `block()`, `WakerBehaviour`, `TickerBehaviour`).
- Ontologie `EmergencyOntology` obligatoire.
- Fonction d'utilité avec justification mathématique.

3 Architecture du système

3.1 Vue d'ensemble

Le système comporte 9 types d'agents sur une grille 50x50. La figure 1 (lien ci-dessous) illustre l'organisation générale.

[Cliquez pour voir le diagramme d'architecture](#)

FIGURE 1 – Architecture globale du SRUU

3.2 Description des agents

- **Agent Capteur (SensorAgent)** : Détecte des incidents périodiquement, envoie `INFORM(HasEmergency)` au répartiteur.
- **Agent Répartiteur (DispatcherAgent)** : Cur décisionnel lance les appels d'offres, évalue les propositions via la fonction d'utilité, assigne et réaffecte.
- **Agent Ambulance** : Gère les missions médicales. Machine à états : `IDLE` → `EN_ROUTE` → `ON_SITE` → `HOSPITAL_TRANSPORT` → `RETURNING` → `IDLE`.
- **Agent Camion de Pompiers** : Ressource eau limitée (150%, seuil critique 20%). Envoie `UNIT_ABORT(LOW_WATER)` puis retour à la base pour `REFILLING`.
- **Agent Police** : Patrouille en `IDLE`, sécurise les périmètres pour `FIRE` et `STRUCTURAL_COLLAPSE`.
- **Agent BCU (BiohazardContainmentUnit)** : Spécialiste `BIOHAZARD` et `CRYOGENIC_LEAK`. Décontamination obligatoire de 10 secondes.
- **Coordinateur médical** : Courtier entre ambulances et hôpitaux (sélection du plus proche avec lits disponibles).
- **Contrôleur de trafic** : Ouvre des corridors d'urgence (timeout 30s).
- **Hôpital** : Gère 50 lits, alerte en cas de saturation (<20%).
- **Enregistreur (Logger)** : Agent passif, génère un journal et un rapport final.

4 Protocoles de communication et ontologie

4.1 FIPA-ACL et Contract Net

Les agents communiquent exclusivement via `ACLMessage`. Le protocole Contract Net est implémenté comme suit :

1. **CFP** : le répartiteur interroge toutes les unités compétentes (recherche DF).
2. **PROPOSE/REFUSE** : les unités répondent avec leur `UnitStatus`.
3. **Évaluation** : calcul de l'utilité (timeout 2s).
4. **ACCEPT_PROPOSAL / REJECT_PROPOSAL** : notification du gagnant et des perdants.
5. **Exécution** : l'unité notifie `MISSION_ARRIVED` et `MISSION_COMPLETE`.

La figure 2 (lien ci-dessous) détaille les échanges.

[Cliquez pour voir le diagramme de séquence du Contract Net](#)

FIGURE 2 – Protocole FIPA Contract Net dans SRUU

Le tableau 1 récapitule les performatives utilisées.

4.2 Ontologie de domaine : EmergencyOntology

L'ontologie définit les concepts suivants :

- `Location(x,y)`
- `Emergency(type, severity, location, id)`
- `UnitStatus(unitId, state, location, workload)`
- `Hospital(name, location, availableBeds, totalBeds)`

et les prédicats : `HasEmergency`, `UnitAvailable`, `RouteCleared`, `HospitalAssignment`.

TABLE 1 – Performatifs FIPA-ACL utilisés

Performatif	Valeur	Usage
INFORM	7	Alerte capteur, état unité, saturation hôpital
REQUEST	18	Demande route, demande hôpital
CFP	34	Appel doffres incident
PROPOSE	11	Offre de service avec statut
ACCEPT_PROPOSAL	36	Acceptation du gagnant
REJECT_PROPOSAL	37	Rejet des autres
REFUSE	10	Refus de participer
FAILURE	12	Échec action, saturation totale

4.3 Découverte de services (DF)

Toute recherche d'agents se fait dynamiquement via le DF, sans noms codés en dur. Exemple :

```

1 DFAgentDescription template = new DFAgentDescription();
2 ServiceDescription sd = new ServiceDescription();
3 sd.setType("FIRE");
4 template.addServices(sd);
5 DFAgentDescription[] results = DFService.search(this, template);

```

5 Fonction d'utilité et logique de décision

5.1 Formulation mathématique

La fonction d'utilité pour une unité u et une urgence e est :

$$U(u, e) = 0.50 \cdot T(u, e) + 0.20 \cdot D(u, e) + 0.15 \cdot W(u) + 0.15 \cdot S(e)$$

avec :

$$T(u, e) = \begin{cases} 1.0 & \text{répondeur primaire} \\ 0.5 & \text{répondeur secondaire} \\ 0.0 & \text{incompétent} \end{cases} \quad (1)$$

$$D(u, e) = \frac{1}{1 + d_{\text{euclidien}}(\text{loc}_u, \text{loc}_e)} \quad (2)$$

$$W(u) = \frac{1}{\text{workload}_u + 1} \quad (3)$$

$$S(e) = \frac{\max(1, \min(5, \text{severity}_e))}{5} \quad (4)$$

Les pondérations sont justifiées dans le tableau 2.

5.2 Algorithme de sélection

1. Filtrer les propositions valides ($T > 0$ et unité non occupée).

TABLE 2 – Pondération de la fonction d'utilité

Critère	Poids	Justification
Correspondance de type	0,50	Critère dominant : compétence technique indispensable
Distance	0,20	Temps de réponse important mais subordonné au type
Charge de travail	0,15	Équilibrage de charge entre unités
Gravité	0,15	Urgence relative sans outrepasser la compétence

2. Si aucun candidat : réessayer ou mettre en file d'attente.
3. Choisir $u^* = \arg \max U(u, e)$.
4. Marquer l'unité comme occupée.
5. Envoyer ACCEPT_PROPOSAL au gagnant, REJECT_PROPOSAL aux autres.

6 Comportement des agents et machines à états finis (FSM)

6.1 Behaviours JADE

Aucune attente active : utilisation exclusive de `CyclicBehaviour`, `TickerBehaviour`, `WakerBehaviour`, `SequentialBehaviour`, `OneShotBehaviour`.

6.2 Machines à états

L'exemple de lambulance est donné sur la figure 3 (lien ci-dessous). Toutes les unités implémentent une FSM similaire.

[Cliquez pour voir la machine à états de lambulance](#)

FIGURE 3 – Machine à états finis simplifiée de lambulance

6.3 Mouvement incrémental

Le déplacement est géré par un `TickerBehaviour` (période 200 ms) :

```

1 int dx = target.getX() - current.getX();
2 int dy = target.getY() - current.getY();
3 if (dx != 0) current.setX(current.getX() + (dx > 0 ? 1 : -1));
4 else if (dy != 0) current.setY(current.getY() + (dy > 0 ? 1 : -1));
5 if (current.equals(target)) onArrival.run();

```

6.4 Bootstrap séquentiel

Le lancement des agents est séquencé avec un délai de 800 ms entre chaque, en utilisant `SequentialBehaviour` et `WakerBehaviour` (pas de `Thread.sleep`).

7 Robustesse et gestion des conflits

- **File d'attente** : lorsqu'aucune proposition n'est reçue après 2 secondes, l'incident est réessayé (2 fois) puis mis en file. Un `TickerBehaviour` périodique (10s) tente de réaffecter les incidents en file.
- **Réaffectation dynamique** : sur réception d'un `UNIT_ABORT` (ex. manque deau), le répartiteur relance immédiatement un `Contract Net` pour cet incident.
- **Prévention de la double affectation** : l'ensemble `busyUnits` est maintenu par le répartiteur ; toute proposition d'une unité déjà occupée est rejetée.
- **Gestion des deadlocks** : timeouts, comportements non bloquants, vérification d'état avant acceptation.

8 Résultats et évaluation

8.1 Scénarios de test

- **Incidents simultanés** : deux capteurs déclenchent `FIRE` et `MEDICAL`. Le répartiteur exécute deux `Contract Nets` parallèles (`conversationId` distincts) et assigne les unités adaptées.
- **Conflit de ressources** : un camion de pompiers manque deau et envoie `UNIT_ABORT`. Le répartiteur réaffecte immédiatement l'incident à une autre unité.
- **Saturation hospitalière** : lorsque les hôpitaux passent sous 20% de lits disponibles, ils émettent une alerte `HOSPITAL_SATURATION`.

8.2 Métriques de performance

Le tableau 3 présente les résultats obtenus lors d'une simulation de test (24 incidents, 120 secondes). Les logs complets sont disponibles sur le dépôt du projet.

TABLE 3 – Métriques par type d'incident (simulation de test)

Type	Temps réponse moy. (s)	Taux résolution	Unités impliquées
FIRE	3,2	85%	FireTruck + Police
MEDICAL	2,9	92%	Ambulance
STRUCTURAL_COLLAPSE	3,8	80%	FireTruck + Police
BIOHAZARD	4,2	75%	BCU
CRYOGENIC_LEAK	3,9	78%	BCU

9 Discussion et évaluation qualitative

9.1 Forces

- Architecture modulaire et extensible.
- Décision multi-critère explicable.
- Bonne tolérance aux indisponibilités transitoires.
- Traçabilité des événements via `LoggerAgent`.

9.2 Limites

- Modèle de trafic simplifié.
- Heuristique de poids statique dans la fonction d'utilité.
- Coordination inter-unités encore principalement centralisée.

9.3 Pistes d'amélioration

- Apprentissage adaptatif des poids d'utilité.
- Intégration de contraintes temporelles réelles (SLA).
- Simulation de dégradations réseau et de pannes partielles.

10 Conclusion

Le projet SRUU montre qu'une architecture multi-agents basée sur JADE et le protocole Contract Net peut répondre efficacement à des incidents hétérogènes en environnement urbain. La combinaison **ontologie + utilité multi-critère + machine à états finis + mécanismes de reprise** permet une allocation cohérente, robuste et justifiable. L'approche reste compatible avec une évolution future vers des politiques d'ordonnement plus avancées et une simulation à plus grande échelle.

A Annexe : Questions/Réponses pour la soutenance

- **Q1 : Architecture fédérée vs centralisée ?** Réponse : architecture hybride le répartiteur centralise la décision, les unités gardent leur autonomie locale.
- **Q2 : Absence d'attente active ?** Réponse : utilisation exclusive des behaviours JADE (`block()`, `WakerBehaviour`).
- **Q3 : Différence CFP vs REQUEST ?** Réponse : CFP pour Contract Net, REQUEST pour requête directe (ex. demandes de route).
- **Q4 : Pourquoi une ontologie ?** Réponse : contrat formel, vérification de type, extensibilité.
- **Q5 : Poids du type (0,50) > distance (0,20) ?** Réponse : primauté fonctionnelle de la compétence.
- **Q6 : Aucune unité primaire disponible ?** Réponse : file d'attente et retry périodique.
- **Q7 : Prévention des deadlocks ?** Réponse : timeouts, `block()`, `busyUnits`, abort explicite.
- **Q8 : ACCEPT_PROPOSAL reçu en état non-IDLE ?** Réponse : rejet immédiat par FAILURE.
- **Q9 : AID vs AgentContainer ?** Réponse : AID = identifiant agent, Container = environnement d'exécution.
- **Q10 : ContentManager : Language + Ontology ?** Réponse : syntaxe (SLCodec) + sémantique.
- **Q11 : Apprentissage par renforcement ?** Réponse : possible via MDP et Q-Learning.
- **Q12 : Adaptation à un graphe routier réel ?** Réponse : remplacer la distance euclidienne par Dijkstra/A* sur un graphe pondéré.

B Glossaire

- **ACL** Agent Communication Language langage standardisé FIPA.
- **AID** Agent Identifier identifiant unique d'un agent.
- **Behaviour** tâche élémentaire exécutée par un agent JADE.
- **CFP** Call For Proposal performatif initiant un appel d'offres.
- **Contract Net** protocole de négociation multi-agents.
- **DF** Directory Facilitator service de pages jaunes de JADE.
- **FIPA** Foundation for Intelligent Physical Agents.
- **FSM** Finite State Machine machine à états finis.
- **JADE** Java Agent DEvelopment Framework.
- **MessageTemplate** filtre de messages JADE.
- **Ontologie** spécification formelle d'une conceptualisation partagée.
- **TickerBehaviour** behaviour périodique.
- **WakerBehaviour** behaviour à déclenchement unique après délai.

C Bibliographie

1. Bellifemine, F., Caire, G., & Greenwood, D. (2007). *Developing Multi-Agent Systems with JADE*. Wiley.
2. FIPA (2002). *FIPA ACL Message Structure Specification*. SC00061G.
3. FIPA (2002). *FIPA Contract Net Interaction Protocol Specification*. XC00025E.
4. Caire, G. (2003). *A Methodology for the Analysis and Design of Multi-Agent Systems using JADE*. Telecom Italia Lab.
5. Odell, J., Parunak, H.V.D., & Bauer, B. (2000). Extending UML for Agents. *Agent-Oriented Information Systems Workshop*.
6. Schoenharl, T., et al. (2006). WIPER : A Multi-Agent System for Emergency Response. *ISCRAM Conference*.
7. Ahmadi, M., et al. (2017). A Multi Agent Based Approach for Prehospital Emergency Management. *Journal of Emergency Medical Services*.
8. JADE Documentation. *Ontology Support in JADE*.